Listing 8.10. The ICM prediction error calculation

```
def ICM(state1, action, state2, forward_scale=1.,
inverse_scale=1e4):
    state1_hat = encoder(state1)
***1***
    state2_hat = encoder(state2)
    state2_hat_pred = forward_model(state1_hat.detach(),
action.detach())  ***2***
    forward_pred_err = forward_scale *
forward_loss(state2_hat_pred, \

state2_hat.detach()).sum(dim=1).unsqueeze(dim=1)
    pred_action = inverse_model(state1_hat, state2_hat)
***3***
    inverse_pred_err = inverse_scale * inverse_loss(pred_action, \
                                        action.detach().flatten())
                                    .unsqueeze(dim=1)
    return forward_pred_err, inverse_pred_err
```

- ***1*** Encodes state1 and state2 using the encoder model
- ***2*** Runs the forward model using the encoded states, but we detach them from the graph
- ***3*** The inverse model returns a softmax probability distribution over actions.

It must be repeated how important it is to properly detach nodes from the graph when running the ICM. Recall that PyTorch (and pretty much all other machine learning libraries) builds a computational graph where nodes are *operations* (computations), and *connections* (also called edges) between nodes are the tensors that flow in and out of individual operations. By calling the `.detach()` method, we disconnect the tensor from the computational graph and treat it just like raw data; this prevents PyTorch from backpropagating through that edge. If we don't detach the `state1_hat` and `state2_hat` tensors when we run the forward model and its loss, the forward model will backpropagate into the encoder and will corrupt the encoder model.

We've now approached the main training loop. Remember, since we're using experience replay, training only happens when we sample from the replay buffer. We'll set up a function that samples from the replay buffer and computes the individual model errors.